

UADE FIGHT

Documento de Diseño del Sistema

Relevamiento y modelo de dominio

MATERIA: Programación Orientada a Objetos (POO)

GRUPO: 6 (Tomás Adrover, Nicolás Pulvirenti, Agustín Chamorro)

AÑO: 2026

1. Descripción general del sistema

UADE FIGHT es un videojuego de rol por turnos (RPG) ambientado en un contexto académico universitario. El jugador arma un equipo de personajes inspirados en estudiantes y debe enfrentarse a enemigos que representan docentes de la universidad.

El sistema se enfoca en tres pilares principales: la gestión de la partida, la administración de personajes y la resolución de combates mediante un sistema de turnos estratégico. El jugador puede iniciar una nueva partida o retomar una previamente guardada, gestionando un equipo con habilidades, equipo y recursos propios.

La aplicación está desarrollada en **Java** con interfaz gráfica en **Swing**. El punto de entrada es la clase `Main`, que inicializa `VentanaPrincipal` y navega entre pantallas mediante `CardLayout`.

1.1 Objetivo del sistema

El objetivo principal es brindar una experiencia de combate estratégico por turnos donde el jugador debe decidir en cada momento cuál es la mejor acción a realizar. Durante cada turno podrá:

- Realizar un ataque básico contra el enemigo.
- Defenderse para reducir el daño recibido.
- Utilizar una habilidad especial de su personaje (cada personaje tiene dos).
- Utilizar un ítem disponible en el inventario compartido (pociones o equipamiento).

A medida que avanza el juego, los personajes recibirán recompensas luego de cada batalla victoriosa, principalmente experiencia, lo que permitirá su crecimiento y evolución. El progreso de los personajes se persiste entre partidas en un archivo separado (`personajes.sav`).

1.2 Alcance funcional

El sistema contempla la gestión completa de una partida, incluyendo:

- Inicio de nueva partida con selección de personajes (Party).

- Guardado y carga de partida existente (`partida.sav`), incluso durante el combate.
- Persistencia de progreso de personajes entre partidas (`personajes.sav`).
- Reinicio completo de stats desde la pantalla de inicio.
- Administración de personajes, clases, atributos y equipo (arma/armadura).
- Gestión del inventario compartido de la Party.
- Desarrollo de batallas por turnos con lógica de velocidad.
- Sistema de experiencia y niveles con mejoras diferenciadas por clase.
- Pantalla de resumen post-batalla con estadísticas, MVP y recompensas de XP.

2. Arquitectura y paquetes

El código se organiza en los siguientes paquetes:

Paquete	Responsabilidad
<code>uadefight</code>	Punto de entrada (<code>Main</code>).
<code>uadefight.modelo</code>	Entidades de dominio: <code>Entidad</code> , <code>Personaje</code> , <code>Party</code> , <code>Batalla</code> , <code>Inventario</code> , enums.
<code>uadefight.modelo.personajes</code>	Subclases jugables: <code>Guerrero</code> , <code>Mago</code> , <code>Arquero</code> , <code>Curador</code> .
<code>uadefight.modelo.enemigos</code>	<code>Enemigo</code> , <code>Profesor</code> , <code>Ayudante</code> , <code>DecisionEnemigo</code> .
<code>uadefight.modelo.items</code>	<code>Item</code> , <code>Pocion</code> , <code>Equipable</code> , <code>Arma</code> , <code>Armadura</code> .
<code>uadefight.modelo.habilidades</code>	<code>Habilidad</code> , <code>TipoEfecto</code> .
<code>uadefight.control</code>	Lógica de negocio: <code>ControladorBatalla</code> , <code>ControladorTurnos</code> , <code>GestorDePartida</code> , <code>GestorDePersonajes</code> .
<code>uadefight.gui</code>	Interfaz: <code>VentanaPrincipal</code> , <code>PantallaInicio</code> , <code>PantallaParty</code> , <code>PantallaBatalla</code> , <code>PantallaResumen</code> .
<code>uadefight.util</code>	Utilidades visuales: <code>Paleta</code> , <code>Sprites</code> , <code>BotonPixel</code> .

3. Entidades principales del dominio

Entidad	Clase	Descripción y Responsabilidad
Punto de entrada	<code>Main</code>	Inicializa Swing y abre <code>VentanaPrincipal</code> .

Entidad	Clase	Descripción y Responsabilidad
Ventana principal	VentanaPrincipal	Contenedor con CardLayout que alterna entre las 4 pantallas del juego (960×640 px).
Partida	GestorDePartida	Guarda y carga el estado de la partida (partida.sav) mediante serialización Java. Contiene EstadoPartida (Party + Batalla opcional).
Progreso de personajes	GestorDePersonajes	Guarda y carga el progreso persistente de personajes (personajes.sav): nivel, XP, stats y equipo.
Batalla	Batalla	Contexto de la pelea: Party, enemigos, estado (EstadoBatalla) y estadísticas (turnos, daño, ítems/habilidades usadas). No ejecuta lógica; solo almacena datos.
Controlador de batalla	ControladorBatalla	Cerebro del combate: ejecuta ataques, defensa, habilidades, ítems, turnos de enemigos, reparte XP y calcula MVP.
Controlador de turnos	ControladorTurnos	Ordena participantes por velocidad y gestiona la cola de turnos con un índice. Rearma la cola al finalizar cada ronda. No existe clase Turno : la cola + índice reemplazan esa abstracción.
Entidad (abstracta)	Entidad	Clase base de Personaje y Enemigo . Atributos: nombre, vida, vida máxima, ataque, defensa, velocidad, flag defendiendo .
Personaje (abstracta)	Personaje	Entidad del jugador. Agrega: maná, nivel, experiencia, lista de Habilidad , Arma y Armadura equipadas.
Enemigo	Enemigo	Entidad controlada por el sistema. Decide acciones vía decidirAccion() y devuelve DecisionEnemigo .
Party	Party	Agrupación entre 2 y 4 personajes aliados. Comparte un único Inventario .
Habilidad	Habilidad	Acción especial con costo de maná, valor de efecto y tipo (DANIO o CURACION).
Item (abstracta)	Item	Base de Poción (consumible) y Equipable (arma/armadura).
Inventario	Inventario	Lista de ítems compartida por la Party.

4. Diagrama de Clases (UML)

4.1 Jerarquía de clases

El diagrama refleja las siguientes jerarquías y relaciones principales:

```
Entidad (abstracta)
├── Personaje (abstracta)
│   ├── Guerrero    → instancia: "Chamo"
│   ├── Curador     → instancia: "Pulvi"
│   ├── Arquero     → instancia: "Toti"
│   └── Mago        → instancia: "Joaco"
└── Enemigo
    ├── Profesor    → instancia: "Godio"
    └── Ayudante     → instancia: "Javier Valda"

Item (abstracta)
├── Pocion (consumible)
└── Equipable (abstracta)
    ├── Arma
    └── Armadura

Batalla —contiene→ Party —contiene→ Inventario
Batalla —contiene→ List<Enemigo>
ControladorBatalla —usa→ Batalla + ControladorTurnos
GestorDePartida.EstadoPartida —contiene→ Party + Batalla
VentanaPrincipal —contiene→ PantallaInicio | PantallaParty | PantallaBatalla |
PantallaResumen
```

Relaciones clave:

- `Personaje` tiene una lista de `Habilidad` y puede equipar `Arma` y `Armadura`.
- `ControladorBatalla` delega el orden de turnos a `ControladorTurnos`.
- `Enemigo.decidirAccion()` devuelve `DecisionEnemigo` (objetivo + si usa especial).
- La GUI (`PantallaBatalla`) delega toda la lógica de combate a `ControladorBatalla`.

4.2 Enums del sistema

Enum	Valores	Uso
<code>EstadoBatalla</code>	<code>EN_CURSO</code> , <code>VICTORIA</code> , <code>DERROTA</code> , <code>PAUSADA</code>	Estado actual de la batalla.

Enum	Valores	Uso
TipoAccion	ATACAR , DEFENDER , USAR_HABILIDAD , USAR_ITEM	Catálogo de acciones posibles en combate. Está definido en el modelo para documentar las operaciones del jugador, pero la GUI (PantallaBatalla) no lo usa directamente: cada botón invoca un método concreto de ControladorBatalla (ejecutarAtaque , ejecutarDefensa , ejecutarHabilidad , ejecutarUsoItem).
TipoEfecto	DANIO , CURACION	Tipo de efecto de una Habilidad .

4.3 Atributos fundamentales de Entidad

Atributo	Tipo	Descripción
nombre	String	Identificador del personaje o enemigo.
vida / vidaMaxima	int	HP actual y máximo. Al llegar a 0, la entidad es derrotada.
ataque	int	Valor base de ataque.
defensa	int	Valor base de defensa.
velocidad	int	Determina el orden de actuación en turnos.
defendiendo	boolean	Si está en guardia (reduce daño recibido 50%).

Atributos adicionales de Personaje :

Atributo	Tipo	Descripción
mana / manaMaximo	int	Puntos de maná. Se consume al usar habilidades.
nivel	int	Nivel actual. Comienza en 1.
experiencia	int	XP acumulada para subir de nivel.
habilidades	List<Habilidad>	Habilidades especiales (2 por personaje).
armaEquipada	Arma	Arma equipada (+ataque).
armaduraEquipada	Armadura	Armadura equipada (+defensa).

Atributos adicionales de Enemigo :

Atributo	Tipo	Descripción
xpQueOtorga	int	Experiencia que otorga al ser derrotado.

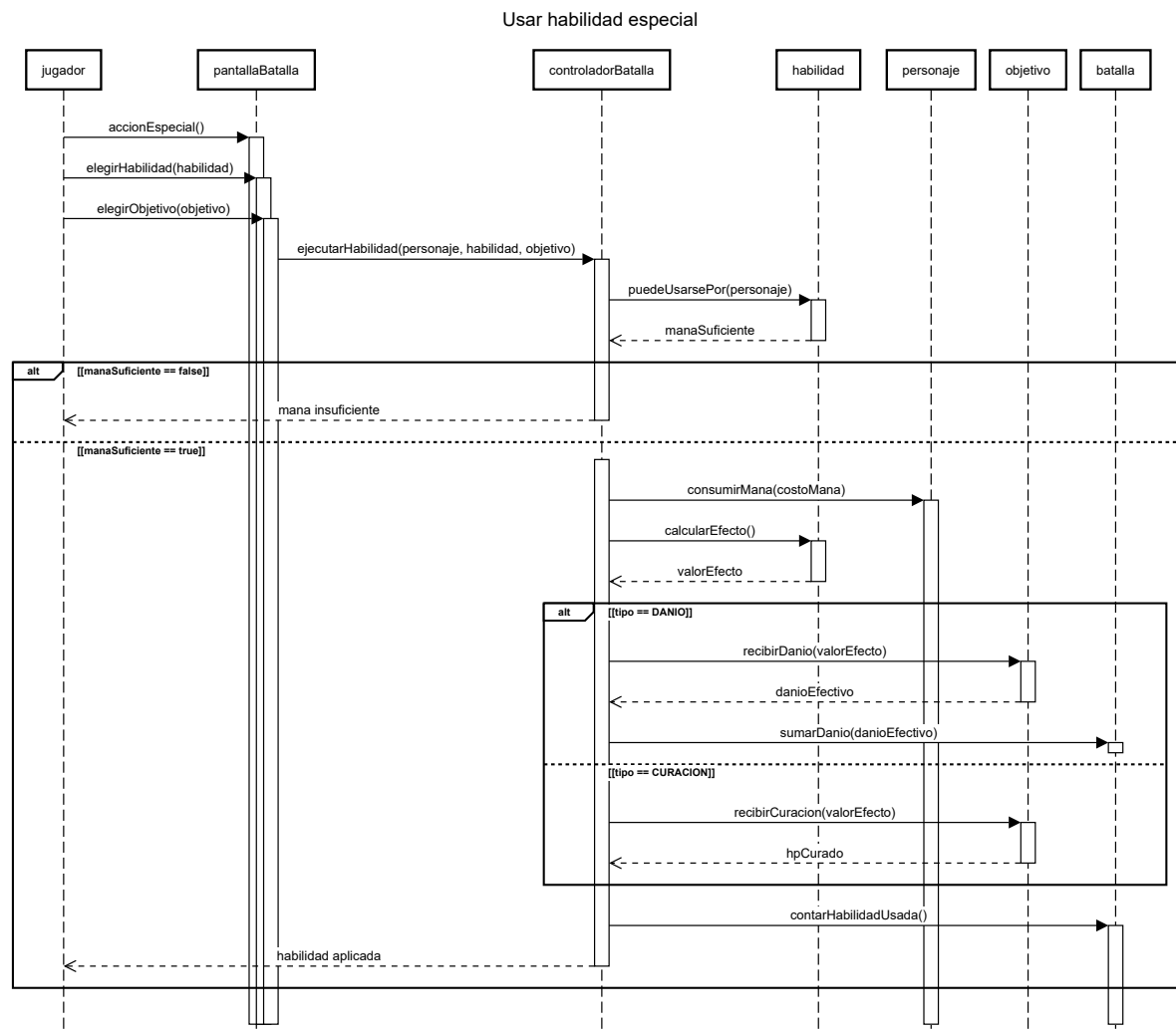
Atributo	Tipo	Descripción
probabilidadEspecial	int	Probabilidad (0–100) de usar ataque especial en su turno.

4.4 Diagramas de secuencia

Los diagramas de secuencia muestran la interacción entre objetos durante acciones concretas del combate. Siguen el mismo criterio que el diagrama de clases: el **Jugador** inicia la acción, la **GUI** (`PantallaBatalla`) recoge la decisión y delega en **ControladorBatalla**, que coordina el resto de participantes.

4.4.1 Usar habilidad especial

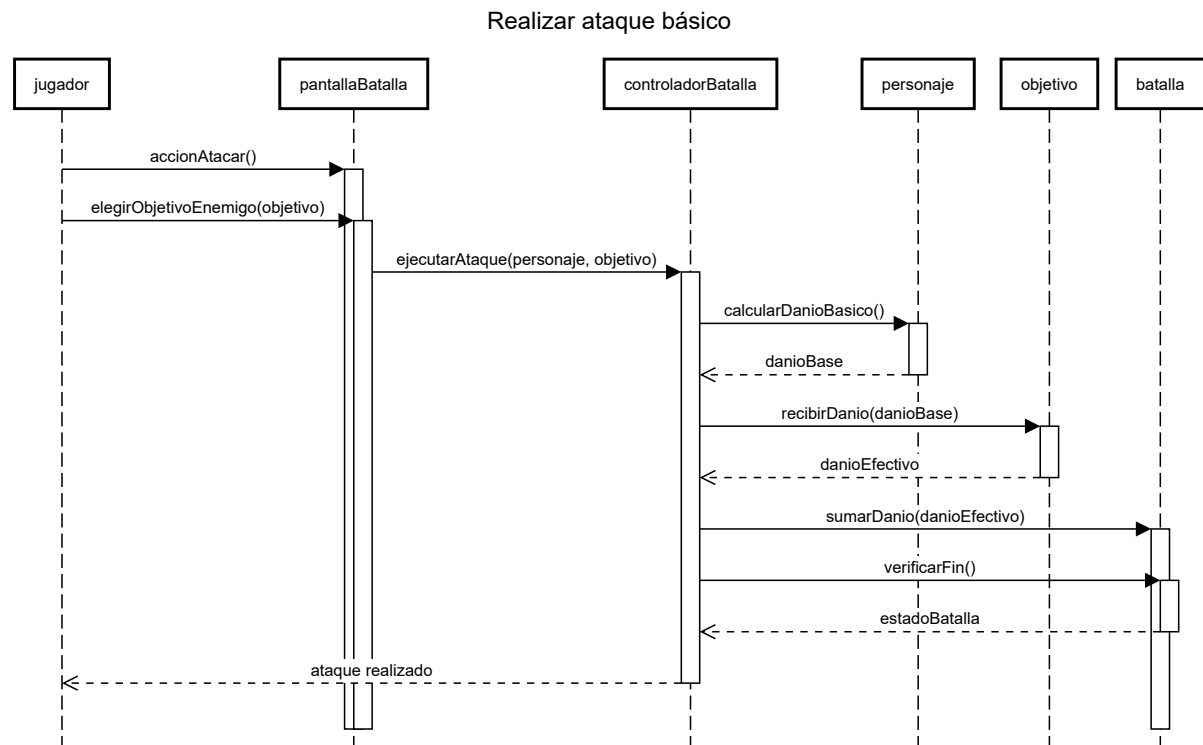
Flujo cuando el jugador elige **ESPECIAL** en su turno: selecciona una habilidad y un objetivo (`Entidad`, aliado o enemigo según el tipo). `ControladorBatalla` valida el maná, consume MP, calcula el efecto y lo aplica como daño o curación.



4.4.2 Realizar ataque básico

Flujo cuando el jugador elige **ATACAR**: selecciona un enemigo como objetivo. `ControladorBatalla` calcula el daño del personaje activo, lo aplica sobre el enemigo y

actualiza las estadísticas de la batalla.



5. Personajes jugables

Cada personaje del juego es una **instancia** de una clase de rol. El nombre visible es el del estudiante; la clase define stats, habilidades y progresión.

Nombre	Clase	Rol	Stats base (HP / MP / ATK / DEF / VEL)
Chamo	Guerrero	Tanque / daño	95 / 30 / 22 / 15 / 10
Pulvi	Curador	Soporte / curación	70 / 70 / 12 / 10 / 11
Toti	Arquero	Velocidad / daño	75 / 45 / 18 / 9 / 18
Joaco	Mago	Daño mágico	60 / 80 / 14 / 6 / 12

5.1 Habilidades por personaje

Cada personaje tiene **dos habilidades** especiales:

Personaje	Habilidad	Costo MP	Efecto	Tipo
Chamo	Espadaso	10	35	Daño
Chamo	Punio fuerte	15	12	Daño
Pulvi	Curación Mayor	18	35	Curación
Pulvi	Besito	10	18	Curación
Toti	Flecha Perforante	14	32	Daño

Personaje	Habilidad	Costo MP	Efecto	Tipo
Toti	Lluvia de Flechas	18	26	Daño
Joaco	Bola de Fuego	20	40	Daño
Joaco	Magia negra	12	28	Daño

El valor de efecto de cada habilidad tiene variación aleatoria del 90% al 110%
(`Habilidad.calcularEfecto()`).

5.2 Mejoras por nivel (por clase)

Al subir de nivel, cada clase aplica mejoras distintas sobrescribiendo `aplicarMejoraPorNivel()`. Las cuatro clases jugables **no llaman a** `super()`, por lo que el maná máximo **no aumenta** al subir de nivel: solo se restaura al valor actual con `restaurarMana(getManaMaximo())`.

Clase	HP máx.	ATK	DEF	VEL	Maná al subir
Guerrero	+12	+4	+3	+1	restaura al máximo actual (no sube el máximo)
Mago	+5	+3	+1	+1	restaura al máximo actual (no sube el máximo)
Arquero	+7	+3	+1	+3	restaura al máximo actual (no sube el máximo)
Curador	+7	+2	+3	+1	restaura al máximo actual (no sube el máximo)

Al subir de nivel, la vida se restaura al nuevo máximo (`setVida(getVidaMaxima())`).

6. Enemigos

Los enemigos son controlados por el sistema. En cada turno, `Enemigo.decidirAccion()` elige un objetivo al azar entre los personajes vivos y determina si usa su ataque especial según `probabilidadEspecial`. La ejecución real del golpe la realiza `ControladorBatalla.ejecutarTurnoEnemigo()`.

Nombre	Clase	Stats base (ATK / DEF / VEL)	XP	Prob. especial	Ataque especial	Multiplicador
Godio	Profesor	18 / 8 / 9	320	25%	Parcial Sorpresa	×1.8

Nombre	Clase	Stats base (ATK / DEF / VEL)	XP	Prob. especial	Ataque especial	Multiplicador
Javier Valda	Ayudante	11 / 5 / 13	120	15%	Cachar copiandose	×1.4

Vida de enemigos (escalada dinámica):

La vida máxima de cada enemigo se calcula al iniciar la batalla según la Party:

$$\text{vida} = \text{base} \times (1 + (\text{nivelPromedio} - 1) \times 0.20) \times (\text{cantidadParty} / 2)$$

- Base de Godio: 120 HP. Base de Javier Valda: 70 HP.
- Con party de 4 personajes nivel promedio 3, los enemigos son considerablemente más resistentes.

Daño enemigo:

El daño escala con el nivel promedio de la Party (+12% por nivel) y con una ventaja por cantidad ($\max(1.2, \text{personajesVivos} / \text{enemigosVivos})$), más variación aleatoria del 85% al 115%.

7. Sistema de combate

7.1 Mecánica de turnos

Los combates se desarrollan mediante un sistema de turnos gestionado por

`ControladorTurnos`:

1. Al iniciar (o al terminar una ronda), todos los participantes vivos se ordenan por velocidad descendente.
2. Se avanza con un índice en la cola. Si el participante está muerto, se saltea.
3. Al llegar al final de la cola, se rearma con los vivos y se reordena por velocidad.
4. Al comenzar cada turno, se desactiva la defensa del participante activo (`desactivarDefensa()`).

Durante cada turno del jugador puede realizarse una de las cuatro acciones del combate (modeladas en el enum `TipoAccion` y ejecutadas vía `ControladorBatalla`):

- **Atacar** (`ATACAR` → `ejecutarAtaque`): inflige daño al objetivo enemigo.
- **Defender** (`DEFENDER` → `ejecutarDefensa`): activa el flag `defendiendo` hasta el próximo turno del personaje.
- **Especial** (`USAR_HABILIDAD` → `ejecutarHabilidad`): usa una de las dos habilidades del personaje (consume maná).
- **Item** (`USAR_ITEM` → `ejecutarUsoItem`): usa un ítem del inventario compartido.

Los turnos de enemigos se resuelven automáticamente con un delay de 450 ms.

7.2 Fórmula de daño (ataque básico)

El daño **no** se calcula como `Ataque - Defensa`. Se usa mitigación porcentual:

```
mitigación = defensaTotal / (defensaTotal + 20)
dañoMitigado = dañoBase × (1 - mitigación)
si defendiendo: dañoMitigado /= 2
dañoEfectivo = max(1, round(dañoMitigado))
```

Donde `dañoBase` se calcula así:

- **Entidad genérica:** `ataqueTotal × random(0.85..1.15)`
- **Personaje:** `ataqueTotal × random(0.85..1.15) × (1 + (nivel - 1) × 0.12)`
- `ataqueTotal` = ataque base + bonus del arma equipada.
- `defensaTotal` = defensa base + bonus de la armadura equipada.

7.3 Acción defender

Cuando un personaje selecciona Defender, el flag `defendiendo` se activa. El daño recibido se reduce a la mitad (además de la mitigación por defensa) hasta el inicio de su próximo turno. No consume maná ni genera daño.

7.4 Condiciones de fin de batalla

`Batalla.verificarFin()` evalúa:

- **Victoria:** todos los enemigos tienen vida ≤ 0 .
- **Derrota:** ningún personaje de la Party está vivo.
- **En curso / Pausada:** en cualquier otro caso.

7.5 Efectos de estado

No hay sistema de efectos temporales (rebote, veneno, robo de maná, etc.) El único estado especial es el flag `defendiendo` en `Entidad`.

8. Inventario e ítems

El inventario es **único y compartido** por toda la Party (`Party.inventario`). Al iniciar cada batalla nueva, la Party recibe los siguientes ítems iniciales:

Tipo	Nombre	Efecto
Poción	Poción Chica	+25 HP
Poción	Poción Mediana	+35 HP
Poción	Poción Grande	+50 HP

Tipo	Nombre	Efecto
Armadura	Armadura UADE	+6 defensa al equiparse
Arma	Espada de Apuntes	+5 ataque al equiparse

8.1 Jerarquía de ítems

```

Item (abstracta)
├── Poción          → esConsumible() = true. Cura HP a un Personaje vivo.
└── Equipable (abstracta)
    ├── Arma        → +ataqueExtra al equiparse en un Personaje.
    └── Armadura     → +defensaExtra al equiparse en un Personaje.

```

- Las pociones se eliminan del inventario al usarse.
- Los equipables también se eliminan del inventario al equiparse (quedan puestos en el personaje).
- Un personaje solo puede tener un arma y una armadura equipadas a la vez; al equipar una nueva, la anterior se desequipa.

Nota: No están implementados elixir, antídoto ni pociones de maná.

9. Sistema de experiencia y niveles

9.1 Obtención de experiencia

Al finalizar una batalla victoriosa, `ControladorBatalla.repartirRecompensa()` suma la XP de todos los enemigos (`Enemigo.xpQueOtorga`) y la divide en partes iguales entre los personajes **vivos** de la Party.

En la batalla estándar (Godio + Javier Valda): **440 XP total** → repartida entre los vivos.

9.2 Subida de nivel

$$\text{XP necesaria para siguiente nivel} = 100 + (\text{nivel} - 1) \times 150$$

Ejemplos:

- Nivel 1 → 2: 100 XP
- Nivel 2 → 3: 250 XP
- Nivel 3 → 4: 400 XP

La experiencia sobrante se conserva. Un personaje puede subir varios niveles de una vez (`Personaje.ganarExperiencia()`).

Además del bonus por nivel de la clase (sección 5.2), los personajes ganan **+12% de daño por cada nivel** en ataques básicos.

9.3 Recompensas de batalla

Al finalizar una batalla ganada:

- **Experiencia:** repartida entre personajes vivos. El progreso se guarda en `personajes.sav`.
- **Ítems aleatorios:** no implementado.
- **Oro:** no implementado.

10. Persistencia de datos

El sistema utiliza **dos archivos** de guardado mediante serialización Java:

Archivo	Gestor	Contenido
<code>partida.sav</code>	<code>GestorDePartida</code>	<code>EstadoPartida</code> : Party completa + Batalla en curso (estado, enemigos, estadísticas). Permite continuar una partida desde la pantalla de inicio o guardar durante el combate.
<code>personajes.sav</code>	<code>GestorDePersonajes</code>	<code>Map<String, Personaje></code> con el progreso persistente (nivel, XP, stats, equipo) de cada personaje por nombre. Se actualiza al ganar una batalla. Se carga al armar la Party.

Desde `PantallaInicio`, el botón **REINICIAR STATS** borra ambos archivos y recrea la pantalla de Party con personajes en nivel 1.

11. Diseño de interfaz de usuario

La ventana principal (`VentanaPrincipal`) usa resolución fija **960×640 px** y navega entre 4 pantallas:

11.1 Pantalla de inicio (`PantallaInicio`)

Pantalla de bienvenida con título "UADE FIGHT", escenario pixel-art (obelisco, edificios, sol) y botones:

- **JUGAR:** navega a selección de Party.
- **CONTINUAR:** carga `partida.sav` (deshabilitado si no existe).
- **REINICIAR STATS:** borra progreso guardado.
- **SALIR:** cierra la aplicación.



Figura 1 — Pantalla de inicio.

11.2 Pantalla de selección de Party (`PantallaParty`)

El jugador arma su equipo eligiendo entre 4 personajes (Chamo, Pulvi, Toti, Joaco). Puede seleccionar entre 2 y 4 personajes.

- Slots superiores (4 posiciones) con sprite, nombre, clase y nivel.
- Galería inferior con tarjetas clickeables que muestran info detallada (stats, daño estimado, habilidades).
- Contador "N / 4 ELEGIDOS".
- Botón **PELEAR** (habilitado con ≥ 2 personajes).
- Botón **VOLVER** al inicio.
- Al entrar, los personajes se curan completamente. Si hay progreso en `personajes.sav`, se recupera nivel y XP.



Figura 2 — Pantalla de armado de Party con 4 slots y galería de personajes disponibles.

11.3 Pantalla de batalla (`PantallaBatalla`)

Pantalla principal del combate. Escenario de aula (pizarra, ventana, tarima).

- Enemigos a la izquierda; Party a la derecha. El participante con turno activo se dibuja más grande.
- Barras de HP para cada luchador; barra de MP para el personaje activo.
- Log de batalla (área de texto con scroll).
- Botones: **ATACAR**, **DEFENSA**, **ITEM**, **ESPECIAL**.
- Botón **GUARDAR** (esquina superior derecha) para guardar `partida.sav`.
- Animaciones simples por acción (salto, temblor, rebote).
- Diálogos de selección para objetivo enemigo/aliado, ítem y habilidad.

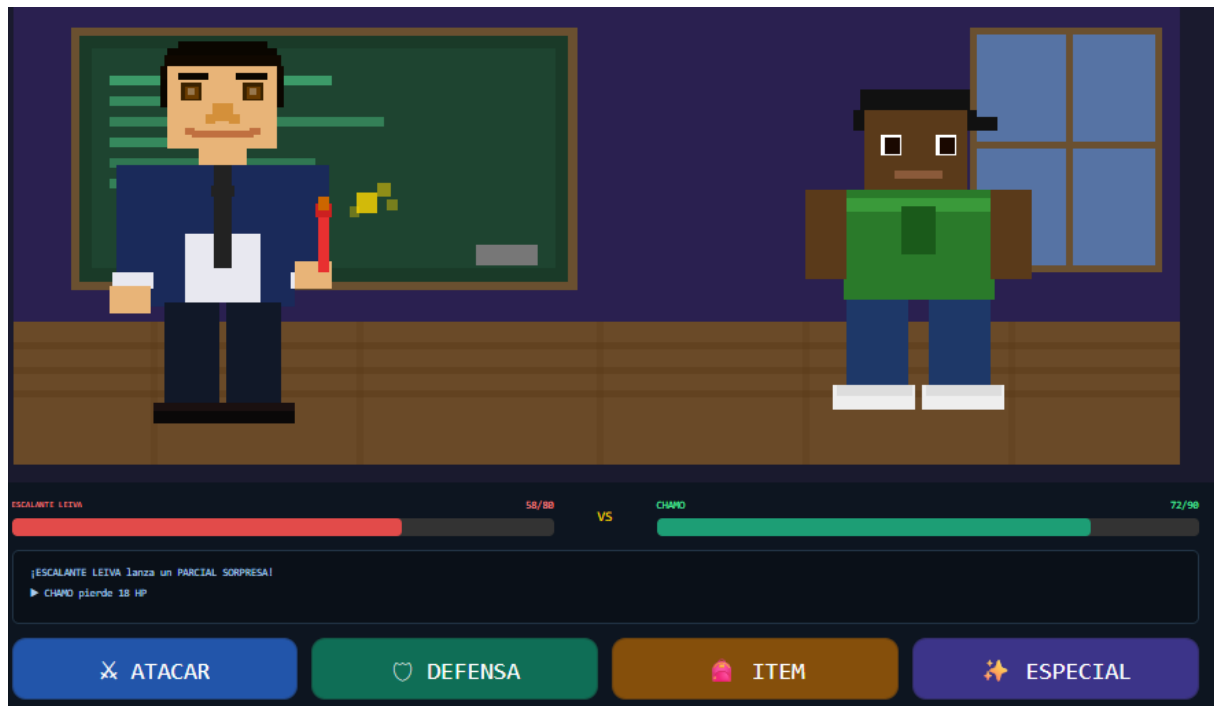


Figura 3 — Pantalla de combate.

11.4 Pantalla de resumen (`PantallaResumen`)

Se muestra al finalizar una batalla.

- Título **VICTORIA** o **DERROTA** (con confeti en victoria).
- Sprites del enemigo derrotado y del MVP.
- Panel de estadísticas: turnos jugados, daño total infligido, ítems usados, habilidades usadas, HP restante de la Party.
- Lista de recompensas de XP por personaje (con indicación de subida de nivel).
- Botones: **VOLVER AL INICIO**, **SALIR**.

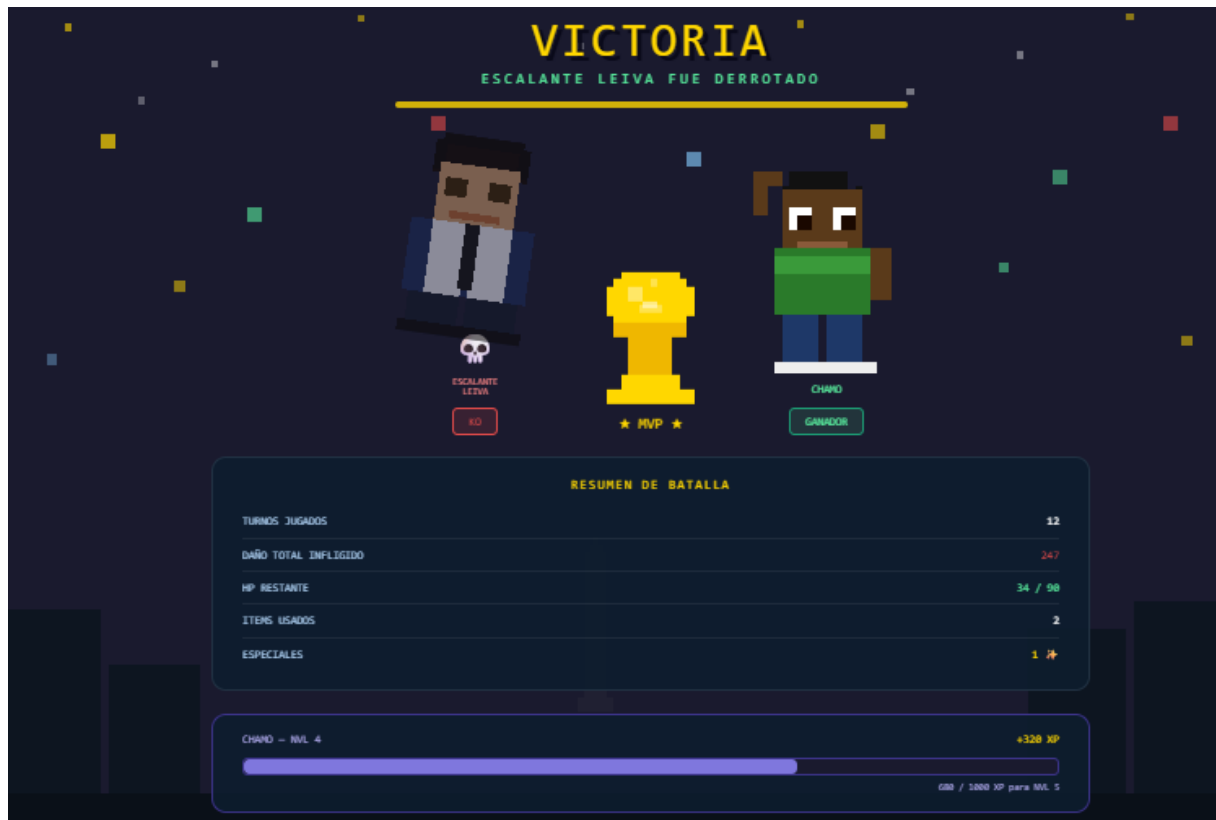


Figura 4 — Pantalla de resumen post-batalla con estadísticas, MVP y barra de progreso de XP.

12. Reglas de negocio

#	Regla
1	El progreso de personajes (nivel, XP, equipo) se conserva en <code>personajes.sav</code> entre partidas.
2	El estado de una partida en curso se conserva en <code>partida.sav</code> .
3	Cada participante ejecuta una sola acción por turno.
4	Las habilidades consumen maná. Si no hay MP suficiente, no se pueden usar (<code>Habilidad.puedeUsarsePor()</code>).
5	Las habilidades de curación solo pueden aplicarse sobre <code>Personaje</code> (aliados).
6	Los ítems deben existir en el inventario. Al usarse, se eliminan del inventario.
7	La acción Defender reduce el daño recibido en un 50% (además de la mitigación por defensa) hasta el próximo turno del personaje.
8	El daño mínimo aplicado siempre es 1.
9	Al finalizar una batalla victoriosa se otorga experiencia a los personajes vivos.
10	La experiencia sobrante tras subir de nivel se acumula para el siguiente.
11	Se puede guardar la partida durante el combate (botón GUARDAR).

#	Regla
12	La Party debe tener entre 2 y 4 personajes para ser válida (<code>Party.esValida()</code>).
13	Un personaje solo puede equipar un arma y una armadura simultáneamente.
14	Los enemigos eligen objetivo al azar entre los personajes vivos.
15	El MVP se calcula como el personaje con mayor <code>ataqueTotal</code> al finalizar la batalla.

13. Flujo general del sistema

Paso	Etapas	Descripción
1	Inicio	<code>Main</code> abre <code>VentanaPrincipal</code> → <code>PantallaInicio</code> . El jugador elige JUGAR, CONTINUAR o REINICIAR STATS.
2	Selección de Party	En <code>PantallaParty</code> , el jugador elige 2–4 personajes. Se cargan stats desde <code>personajes.sav</code> si existen.
3	Inicio de Batalla	Se crea <code>Batalla</code> con Party + enemigos (Godio y Javier Valda). Se agregan ítems iniciales al inventario. <code>ControladorBatalla</code> inicializa <code>ControladorTurnos</code> .
4	Desarrollo del Combate	Por turnos: jugador elige acción o enemigo actúa automáticamente. <code>ControladorBatalla</code> aplica efectos y actualiza log/estadísticas.
5	Fin de Batalla	<code>Batalla.verificarFin()</code> detecta victoria o derrota. Se muestra <code>PantallaResumen</code> .
6	Recompensas	Si victoria: XP repartida, progreso guardado en <code>personajes.sav</code> .
7	Continuación	El jugador vuelve al inicio. Puede guardar partida en cualquier momento durante el combate.

14. Actores del sistema

14.1 Jugador

- Inicia, continúa o reinicia partidas.
- Selecciona los personajes de su equipo.
- Selecciona acciones, objetivos, habilidades e ítems durante los combates.
- Guarda el progreso de la partida en combate.

14.2 Sistema

- Controla el comportamiento de los enemigos (`Enemigo.decidirAccion()` + `ControladorBatalla`).
 - Gestiona el orden y desarrollo de los turnos (`ControladorTurnos`).
 - Calcula el daño, aplica curaciones y gestiona equipo (`ControladorBatalla` + `Entidad` / `Personaje`).
 - Administra la experiencia y las recompensas.
 - Persiste partidas (`GestorDePartida`) y progreso de personajes (`GestorDePersonajes`).
-